

DATA STRUCTURES & ALGORITHMS for AI

**2025 fall semester course for
Bachelor in AI at VU Amsterdam**

Lectures by Hannah Santa Cruz Baur. Based on *Introduction to Algorithms*, Cormen et al.

Course Topics

- Basic data structures
 - Basic sorting algorithms
 - Basic algorithm analysis
 - Hashing
 - Linear sorting
 - Binary search trees
 - Heaps
 - Elementary graph algorithms
 - Minimum spanning trees
 - Shortest path algorithms
 - Matrix operation algorithms
- Time permitting
- Clustering
 - Gradient descent

Course Structure

- Two weekly lectures, each 1 hour 45 min.
 - Monday lecture at 15:30 in MF-FG1.
 - Wednesday lecture at 9:00 in MF-FG2.
- One weekly tutorial session, 1 hour 45 min long.
 - Fridays at 15:30.
 - See Canvas for tutorial group and location information.

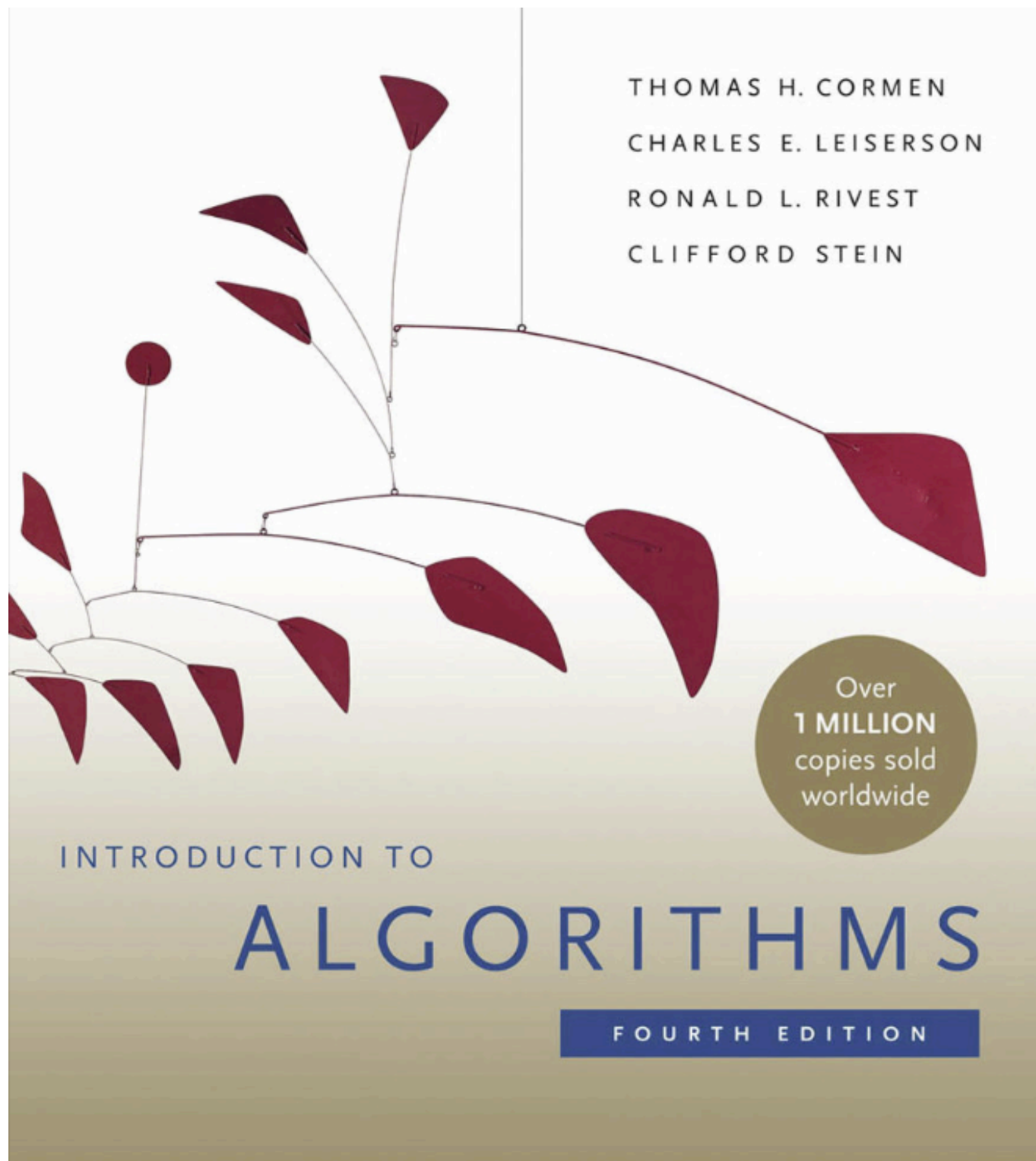
Method of assessment

- Two exams, each worth 35% of your grade.
 - First exam on September 22.
 - Second exam on October 23.
- Four assignments, worth 5% - 5% - 10% - 10% respectively.
Assignments to be handed in after tutorials on:
 - September 12, September 26, October 3, October 10.

Assignment grading

- Honor system - you are expected to use AI to improve your understanding of the subject, not solve the problems.
- Peer review system - you will be reviewing the assignments for one your classmates. Your review should focus on leaving valuable feedback. The clarity and thought of your feedback will count towards 2 out of 10 points for your assignment grade.
- Your assignments will be graded by the TA's you meet in the tutorial sessions.

Literature



We will cover the following:

- Part I- Foundations:
Chapters 1-3
- Part II-Sorting:
Chapters 6-8
- Part III- Data Structures:
Chapters 10-12
- Part VI-Graph algorithms:
Chapters 20-23
- Part VII-Selected topics:
Chapters 28,33

Questions?



Please fill out this form

INTRODUCTION TO ALGORITHMS AND DATA STRUCTURES

Lecture I for course in
Data Structures & Algorithms for AI

Lectures by Hannah Santa Cruz Baur. Based on *Introduction to Algorithms*, Cormen et al.

Algorithms: input and output

- An **algorithm** is a computational procedure which takes an **input** and through a sequence of steps transforms it into an **output**.

Algorithms: input and output

- An **algorithm** is a computational procedure which takes an **input** and through a sequence of steps transforms it into an **output**.
- For example a sorting algorithm may take the sequence:

31, 61, 52, 34, 82

and output the sorted sequence:

31, 34, 51, 61, 82.

Algorithms: input and output

- An **algorithm** is a computational procedure which takes an **input** and through a sequence of steps transforms it into an **output**.
- For example a sorting algorithm may take the sequence:

31, 61, 52, 34, 82

and output the sorted sequence:

31, 34, 51, 61, 82.

- This input sequence is called an **instance** of the sorting problem.

Correct algorithms

- A **problem instance** is a specific input to a computational problem, containing the necessary data for a solution and satisfying any specified constraints.

Correct algorithms

- A **problem instance** is a specific input to a computational problem, containing the necessary data for a solution and satisfying any specified constraints.
- A **correct algorithm** will **halt** (finish in finite time) given any problem instance, and in fact produce the **correct output**.
- An incorrect algorithm, may on some instances
 - never halt
 - halt but produce an incorrect answer.

What is your favorite
computational problem?

Arrays

- An array is stored as a contiguous sequence of bytes in memory.

0	1	2	3	4	5
1	4	2	5	3	6

Arrays

- An array is stored as a contiguous sequence of bytes in memory.

0	1	2	3	4	5
1	4	2	5	3	6

- In this example, the first element of the array has index 0 . Assuming the first element is stored at address a , and each element occupies b bytes of memories, the i 'th element will occupy bytes:

$$a + b(i) \text{ through } a + b(i+1) - 1$$

Arrays

- An array is stored as a contiguous sequence of bytes in memory.
- Assuming the first element has index 0 and is stored at address a , and each element occupies b bytes of memories, the i 'th element will occupy bytes:

$$a + b(i) \text{ through } a + b(i+1) - 1$$

Arrays

- An array is stored as a contiguous sequence of bytes in memory.
- Assuming the first element has index 0 and is stored at address a , and each element occupies b bytes of memories, the i 'th element will occupy bytes:

$$a + b(i) \text{ through } a + b(i+1) - 1$$

- If the first element's address is 1000 , and each element occupies 4 bytes, then:

Element 0 is stored at $1000 + 0 \times 4 = 1000$

Element 1 is stored at $1000 + 1 \times 4 = 1004$

Element 2 is stored at $1000 + 2 \times 4 = 1008$

Arrays

- An array is stored as a contiguous sequence of bytes in memory.

1	2	3	4	5	6
1	4	2	5	3	6

- What bytes will the i 'th element occupy if the first element has index 1 ?

Arrays

- An array is stored as a contiguous sequence of bytes in memory.
- The computer can locate any element, given the address of the first element, a , it's index, s , and the element size, b :

The element i is stored at bytes: $a+b(i-s)$ through $a+b(i-s-1)-1$.

- It takes constant time for the computer to access any element of the array given this formula.

Arrays

- An array is stored as a contiguous sequence of bytes in memory.
- Given elements of a fixed size, it takes the computer constant time to access any element of an array.

Arrays

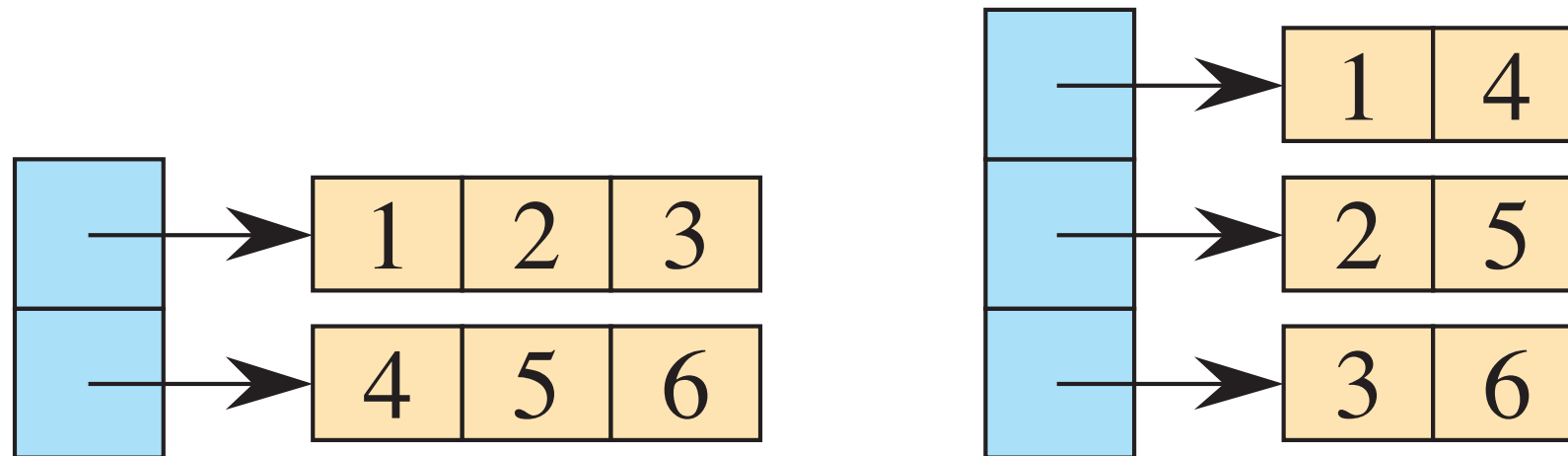
- An array is stored as a contiguous sequence of bytes in memory.
- Given elements of a fixed size, it takes the computer constant time to access any element of an array.
- Given elements of different size, a programming language will store pointers to the elements in the array.
- A pointer will store the address of the element, rather than the value.

Arrays

- An array is stored as a contiguous sequence of bytes in memory.
- Given elements of a fixed size, it takes the computer constant time to access any element of an array.
- Given elements of different size, a programming language will store pointers to the elements in the array.
- A pointer will store the address of the element, rather than the value.
- The size of an address fixed, it is independent of the size of the corresponding element. Thus the computer can access the pointers in constant time, and then follow the pointer to the actual element.

Matrices

- We can store a matrix $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$ in an array in many ways:



- Here we use **arrays of pointers**, which each point to an array. These **stored arrays** could either be the rows (left) or the columns (right).

Matrices

- We can store a matrix $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$ in an array in many ways:

1	2	3	4	5	6
---	---	---	---	---	---

1	4	2	5	3	6
---	---	---	---	---	---

- Here we store the matrix as an array directly, using the common **row-major order** (left) and **column-major order** (right).
- Storing a matrix as an array directly, is usually more efficient.

Dynamic sets

- In a dynamic set, elements are not fixed, but can change over time through insertion and deletions.

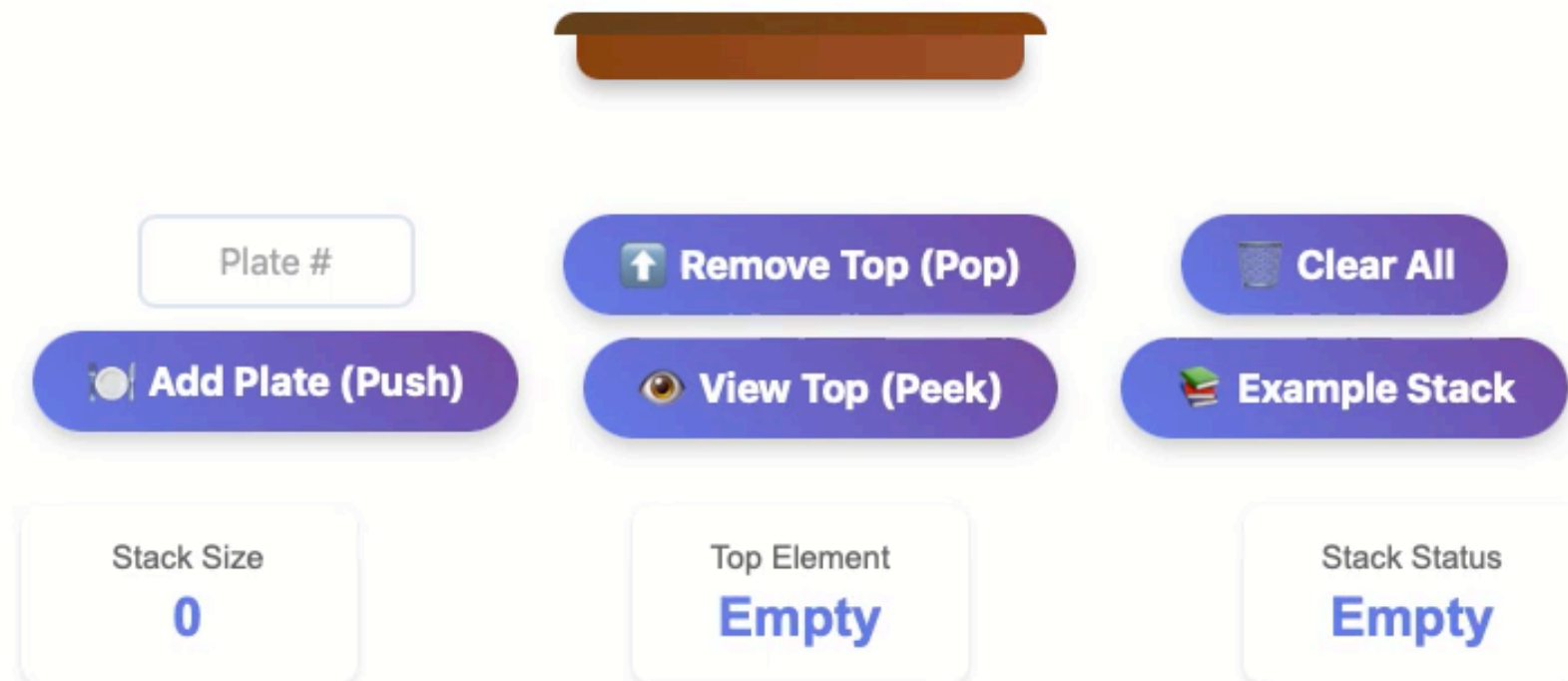
Dynamic sets

- In a dynamic set, elements are not fixed, but can change over time through insertion and deletions.
- A **stack** is a dynamic set, where the delete operation eliminates the most recently inserted element.
- We say stacks have a “last-in, first-out” (LIFO) policy.



Stack of Plates

Interactive LIFO (Last In, First Out) Visualization



Credit: [claude.ai](#) generated visualization

Dynamic sets

- In a dynamic set, elements are not fixed, but can change over time through insertion and deletions.
- A **stack** is a dynamic set, where the delete operation eliminates the most recently inserted element.
- We say stacks have a “last-in, first-out” (LIFO) policy.
- The deletion operation of a stack is called **Pop**, and the insertion operation is called **Push**.

Dynamic sets

- In a dynamic set, elements are not fixed, but can change over time through insertion and deletions.
- A **stack** is a dynamic set, where the delete operation eliminates the most recently inserted element.
- We say stacks have a “last-in, first-out” (LIFO) policy.
- The deletion operation of a stack is called **Pop**, and the insertion operation is called **Push**.
- One example of a pop operation is the **Ctrl+Z** function.

Dynamic sets

- In a dynamic set, elements are not fixed, but can change over time through insertion and deletions.
- A **queue** is a dynamic set, where the delete operation eliminates the element which has been stored the longest.
- We say queues have a “first-in, first-out” (FIFO) policy.

Dynamic sets

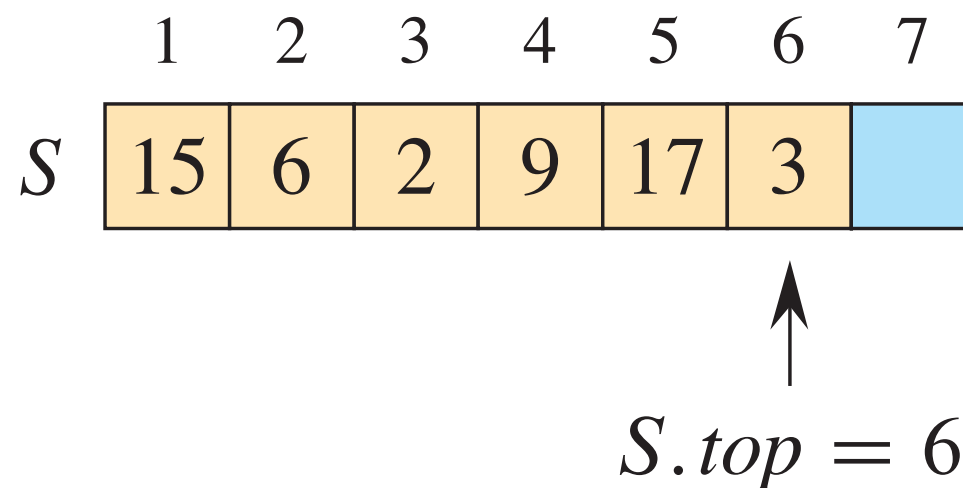
- In a dynamic set, elements are not fixed, but can change over time through insertion and deletions.
- A **queue** is a dynamic set, where the delete operation eliminates the element which has been stored the longest.
- We say queues have a “first-in, first-out” (FIFO) policy.
- A queue has an **head** and a **tail**. When an element is inserted in a queue, we say it is **enqueued** - added to the tail of the queue. When an element is deleted, it is the element at the head of the line, we say it is **dequeued**

Dynamic sets

- In a dynamic set, elements are not fixed, but can change over time through insertion and deletions.
- A **queue** is a dynamic set, where the delete operation eliminates the element which has been stored the longest.
- We say stacks have a “first-in, first-out” (FIFO) policy.
- A queue has an **head** and a **tail**. When an element is inserted in a queue, we say it is **enqueued** - added to the tail of the queue. When an element is deleted, it is the element at the head of the line, we say it is **dequeued**
- Customers waiting in line is the classical queue example.

Stacks as Arrays

- We can implement a stack of at most n elements, with an array $S[1:n]$, by defining attributes $S.top$ and $S.size$.
- $S.top$ is used to refer to the index of the most recently inserted element. The stack then corresponds to the elements of the array $S[1:S.top]$.



Stacks as Arrays

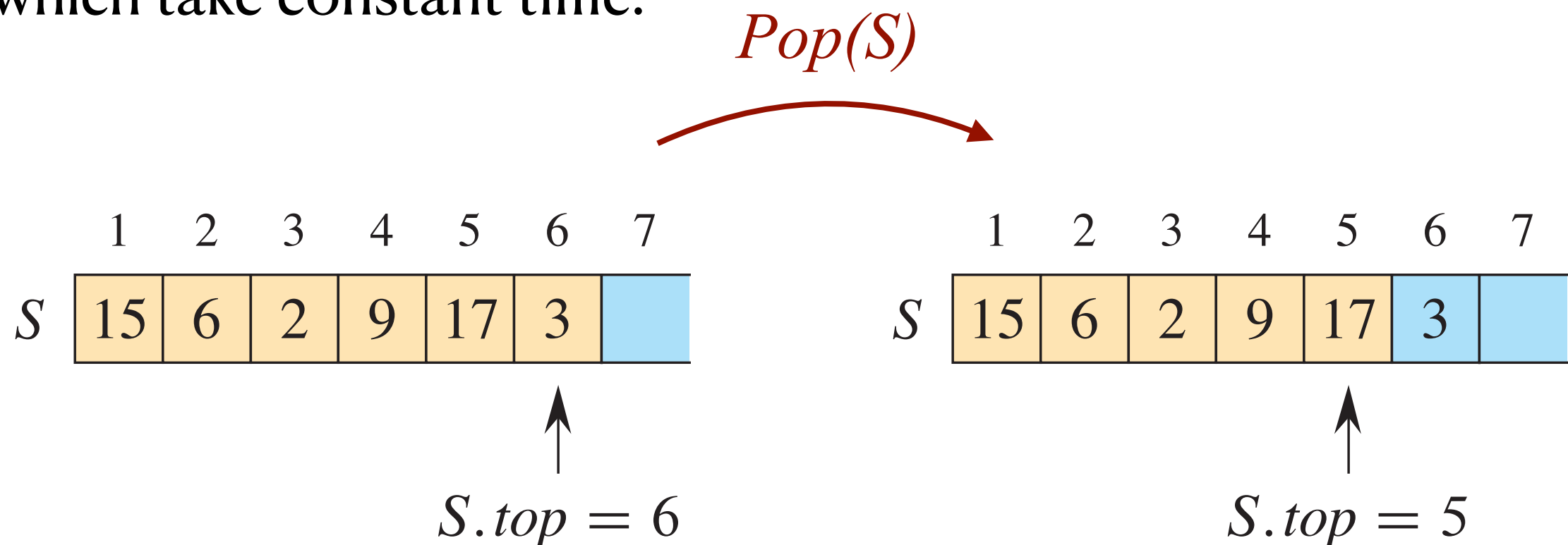
- We can implement a stack of at most n elements, with an array $S[1:n]$, by defining attributes $S.top$ and $S.size$.
- $S.top$ is used to refer to the index of the most recently inserted element. The stack then corresponds to the elements of the array $S[1:S.top]$.

Stacks as Arrays

- We can implement a stack of at most n elements, with an array $S[1:n]$, by defining attributes $S.top$ and $S.size$.
- $S.top$ is used to refer to the index of the most recently inserted element. The stack then corresponds to the elements of the array $S[1:S.top]$.
- $S.size$ refers to the size n of the array. We say the stack **overflows** if $S.top$ exceeds $S.size$.
- We say the stack is empty if $S.top=0$. We say a stack **underflows** when attempting to pop an empty stack.

Stacks as Arrays

- We can implement a stack of at most n elements, with an array $S[1:n]$, by defining attributes $S.top$ and $S.size$.
- We can then implement Pop and Push operations for the stack which take constant time.



Stacks as Arrays

- We can implement a stack of at most n elements, with an array $S[1:n]$, by defining attributes $S.top$ and $S.size$.
- We can then implement push and pop operation for the stack which take constant time.

POP(S)

```
1  if STACK-EMPTY( $S$ )  
2      error “underflow”  
3  else  $S.top = S.top - 1$   
4      return  $S[S.top + 1]$ 
```


Stacks as Arrays

- We can implement a stack of at most n elements, with an array $S[1:n]$, by defining attributes $S.top$ and $S.size$.
- We can then implement push and pop operation for the stack which take constant time.

POP(S)

```
1  if STACK-EMPTY( $S$ )
2      error “underflow”
3  else  $S.top = S.top - 1$ 
4      return  $S[S.top + 1]$ 
```

STACK-EMPTY(S)

```
1  if  $S.top == 0$ 
2      return TRUE
3  else return FALSE
```


Stacks as Arrays

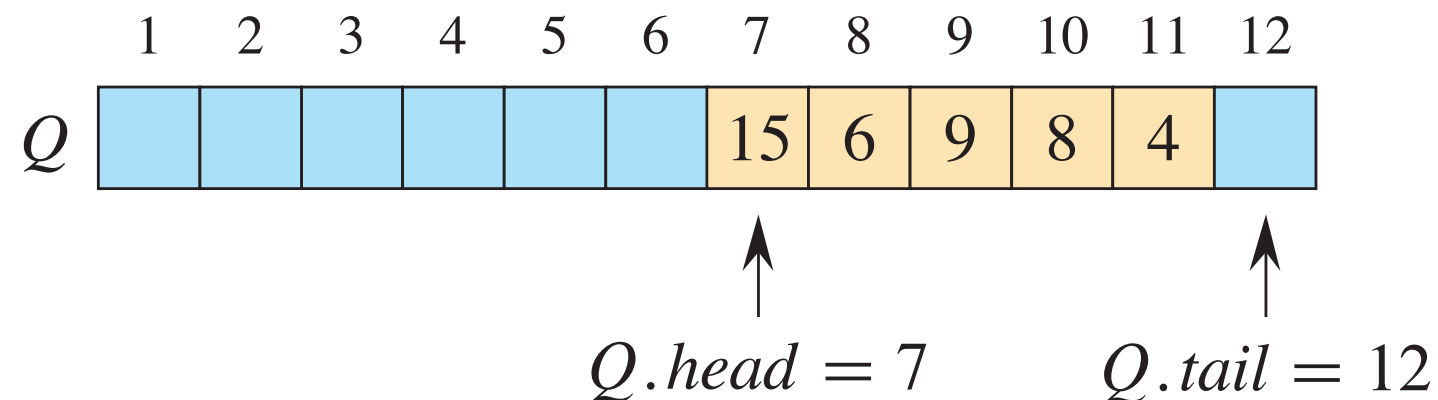
- We can implement a stack of at most n elements, with an array $S[1:n]$, by defining attributes $S.top$ and $S.size$.
- We can then implement push and pop operation for the stack which take constant time.

PUSH(S, x)

```
1  if  $S.top == S.size$   
2      error “overflow”  
3  else  $S.top = S.top + 1$   
4       $S[S.top] = x$ 
```

Queues as Arrays

- We can implement a queue of at most $n-1$ elements, with an array $Q[1:n]$, by defining attributes $Q.head$, $Q.tail$ and $Q.size$.
- $Q.head$ and $Q.tail$ refer to the indexes of the head and of the **future** tail respectively. The queue then corresponds to the elements of the array $Q[Q.head : Q.tail - 1]$, assuming index 1 immediately follows index n in a circular order.



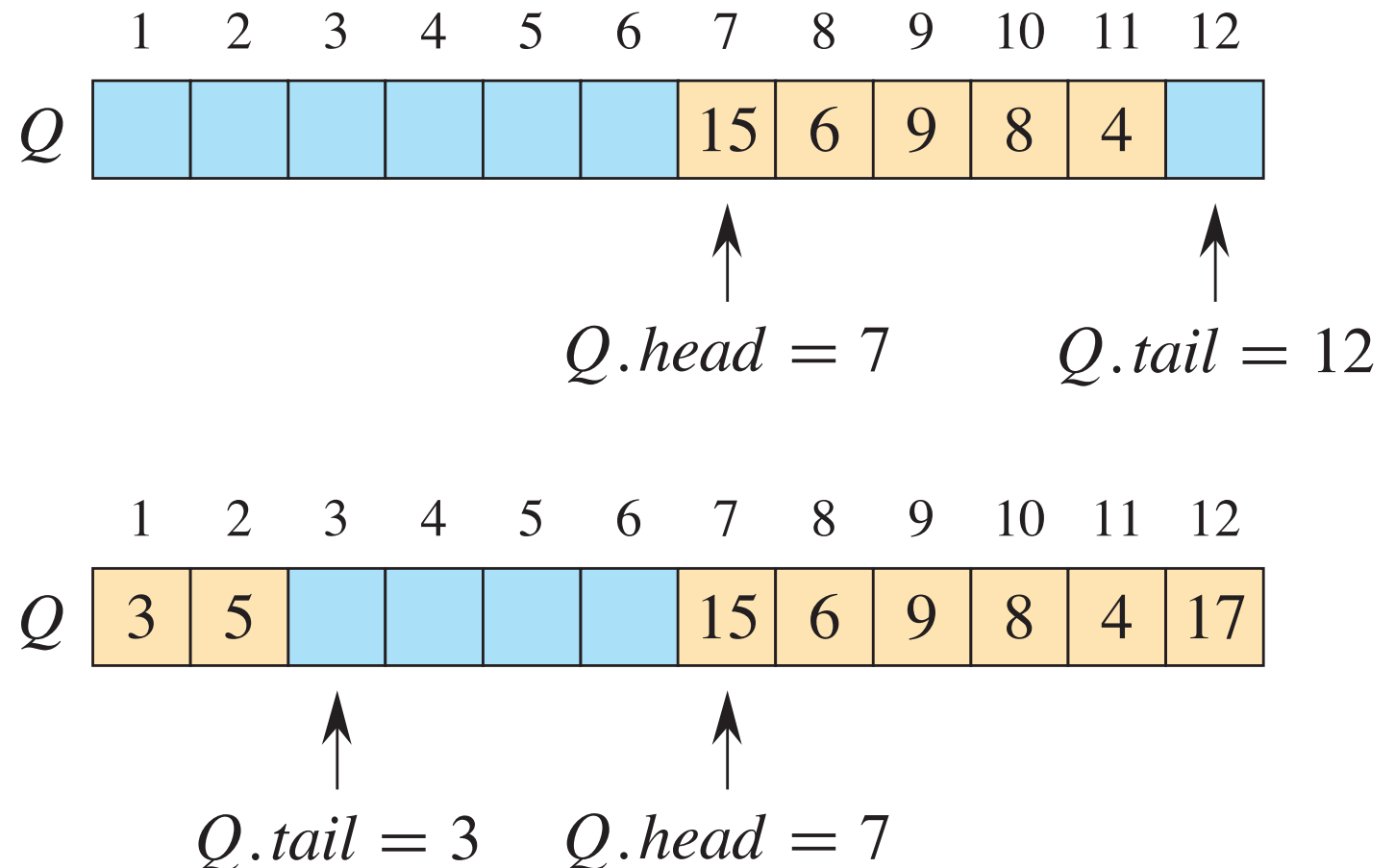
Queues as Arrays

- We can implement a queue of at most $n-1$ elements, with an array $Q[1:n]$, by defining attributes $Q.head$, $Q.tail$ and $Q.size$.
- We can then implement the Enqueue operation:

Enqueue(Q, 17)

Enqueue(Q, 3)

Enqueue(Q, 5)



Queues as Arrays

- We can implement a queue of at most $n-1$ elements, with an array $Q[1:n]$, by defining attributes $Q.head$, $Q.tail$ and $Q.size$.
- We can then implement the Enqueue operation:

ENQUEUE(Q, x)

1 $Q[Q.tail] = x$

2 **if** $Q.tail == Q.size$

3 $Q.tail = 1$

4 **else** $Q.tail = Q.tail + 1$

Queues as Arrays

- We can implement a queue of at most $n-1$ elements, with an array $Q[1:n]$, by defining attributes $Q.head$, $Q.tail$ and $Q.size$.
- We can then implement the Enqueue operation:

ENQUEUE(Q, x)

```
1   $Q[Q.tail] = x$   
2  if  $Q.tail == Q.size$   
3       $Q.tail = 1$   
4  else  $Q.tail = Q.tail + 1$ 
```

How to check for overflow?

Queues as Arrays

- We can implement a queue of at most $n-1$ elements, with an array $Q[1:n]$, by defining attributes $Q.head$, $Q.tail$ and $Q.size$.
- We can then implement the Enqueue operation:

ENQUEUE(Q, x)

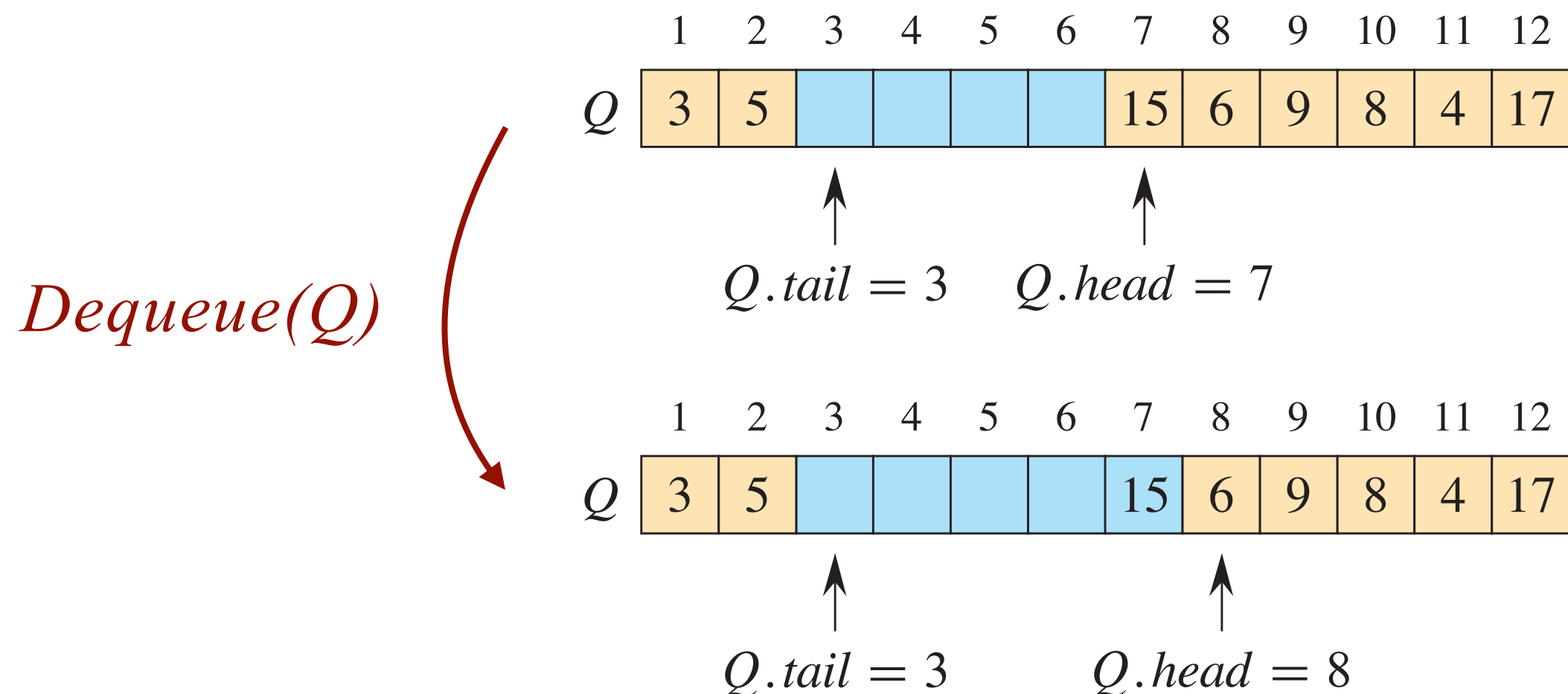
```
1   $Q[Q.tail] = x$ 
2  if  $Q.tail == Q.size$ 
3       $Q.tail = 1$ 
4  else  $Q.tail = Q.tail + 1$ 
```

How to check for overflow?

If $Q.head = Q.tail + 1$
or $Q.head = 1$ & $Q.tail = Q.size$

Queues as Arrays

- We can implement a queue of at most $n-1$ elements, with an array $Q[1:n]$, by defining attributes $Q.head$, $Q.tail$ and $Q.size$.
- We can also implement the Dequeue operation:



Queues as Arrays

- We can implement a queue of at most $n-1$ elements, with an array $Q[1:n]$, by defining attributes $Q.head$, $Q.tail$ and $Q.size$.
- We can also implement the Dequeue operation:

DEQUEUE(Q)

```
1   $x = Q[Q.head]$ 
2  if  $Q.head == Q.size$ 
3       $Q.head = 1$ 
4  else  $Q.head = Q.head + 1$ 
5  return  $x$ 
```


Queues as Arrays

- We can implement a queue of at most $n-1$ elements, with an array $Q[1:n]$, by defining attributes $Q.head$, $Q.tail$ and $Q.size$.
- We can also implement the Dequeue operation:

DEQUEUE(Q)

```
1   $x = Q[Q.head]$ 
2  if  $Q.head == Q.size$ 
3       $Q.head = 1$ 
4  else  $Q.head = Q.head + 1$ 
5  return  $x$ 
```

How to check for underflow?

Queues as Arrays

- We can implement a queue of at most $n-1$ elements, with an array $Q[1:n]$, by defining attributes $Q.head$, $Q.tail$ and $Q.size$.
- We can also implement the Dequeue operation:

DEQUEUE(Q)

```
1   $x = Q[Q.head]$ 
2  if  $Q.head == Q.size$ 
3       $Q.head = 1$ 
4  else  $Q.head = Q.head + 1$ 
5  return  $x$ 
```

How to check for underflow?

If $Q.head = Q.tail$

Exercices

- Explain how to implement two stacks as one array of size n , so that neither stack overflows, unless the total number of elements exceeds n . Implement Push and Pop operations that run in constant time.
- Show how to implement a queue using two stacks.

Exercices

- Explain how to implement two stacks as one array of size n , so that neither stack overflows, unless the total number of elements exceeds n . Implement Push and Pop operations that run in constant time.
Have one stack grow left to right (starting at 1), and the other right to left (starting at n)
- Show how to implement a queue using two stacks.

Exercices

- Explain how to implement two stacks as one array of size n , so that neither stack overflows, unless the total number of elements exceeds n . Implement Push and Pop operations that run in constant time.
 - Have one stack grow left to right (starting at 1), and the other right to left (starting at n)
- Show how to implement a queue using two stacks.
 - Enqueue pushes into “in-stack”
 - Dequeue pops elements from “out-stack”, unless...

Exercices

- Explain how to implement two stacks as one array of size n , so that neither stack overflows, unless the total number of elements exceeds n . Implement Push and Pop operations that run in constant time.

Have one stack grow left to right (starting at 1),
and the other right to left (starting at n)

- Show how to implement a queue using two stacks.
Enqueue pushes into “in-stack”
Deque pop elements from “out-stack”.

Unless “out-stack” is empty, in which case:
it Pops everything from “in-stack” and Pushes into “out-stack” first.

Homework (due with Assignment 1)

- Show how to implement a stack using two queues.